

Software Engineering

From coding software to engineering reliable systems

Module2: Software requirement analysis and specification

Dr. Vivek Kumar Singh



Department of Information Technology

ABV-IIITM Gwalior

(Email : vks@iiitm.ac.in)

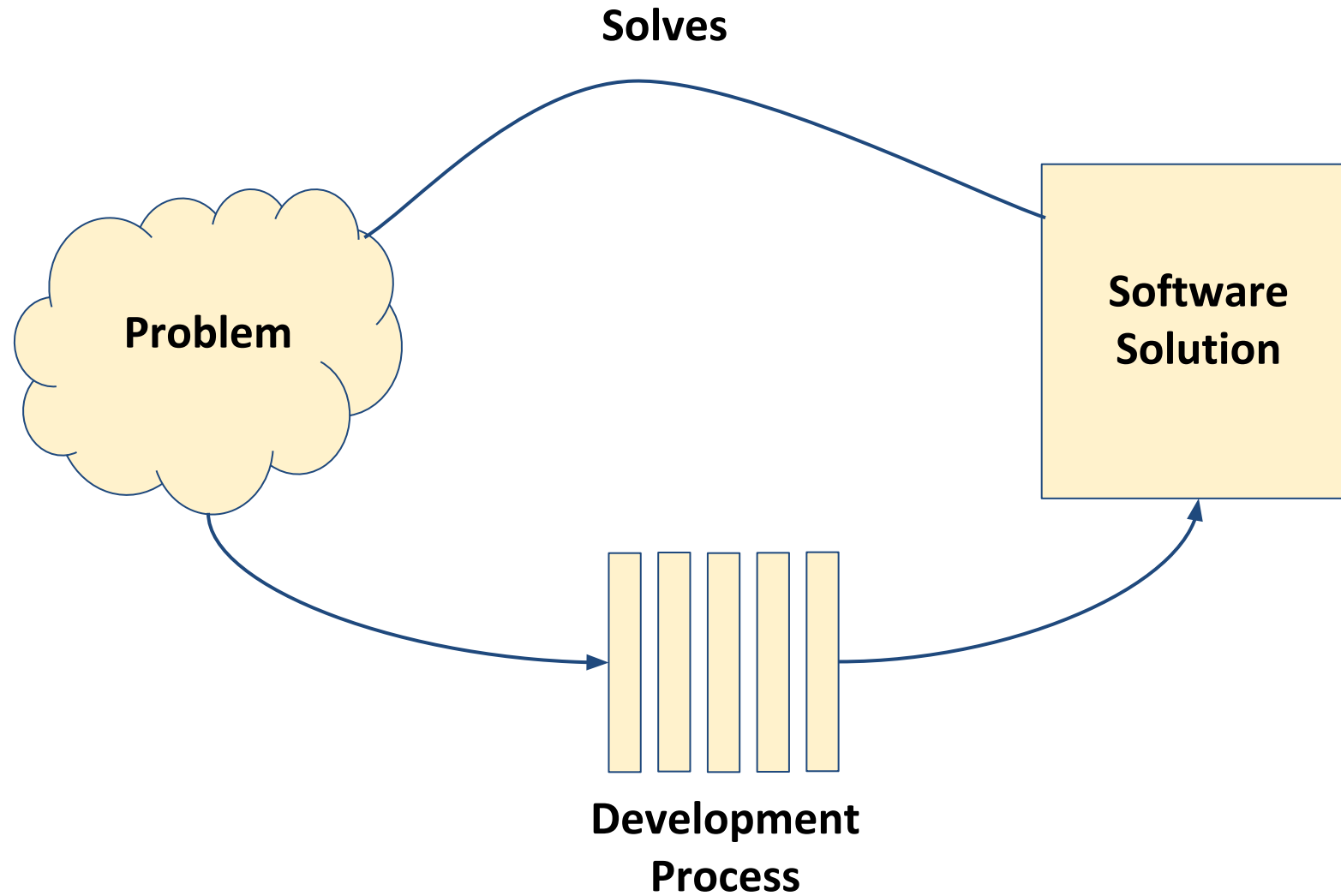
Contents:

- ☐ Value and need of good SRS
- ☐ Requirement process,
- ☐ Requirement specification,
- ☐ Functional specifications with use-cases,
- ☐ Other approaches for analysis,
- ☐ Validation

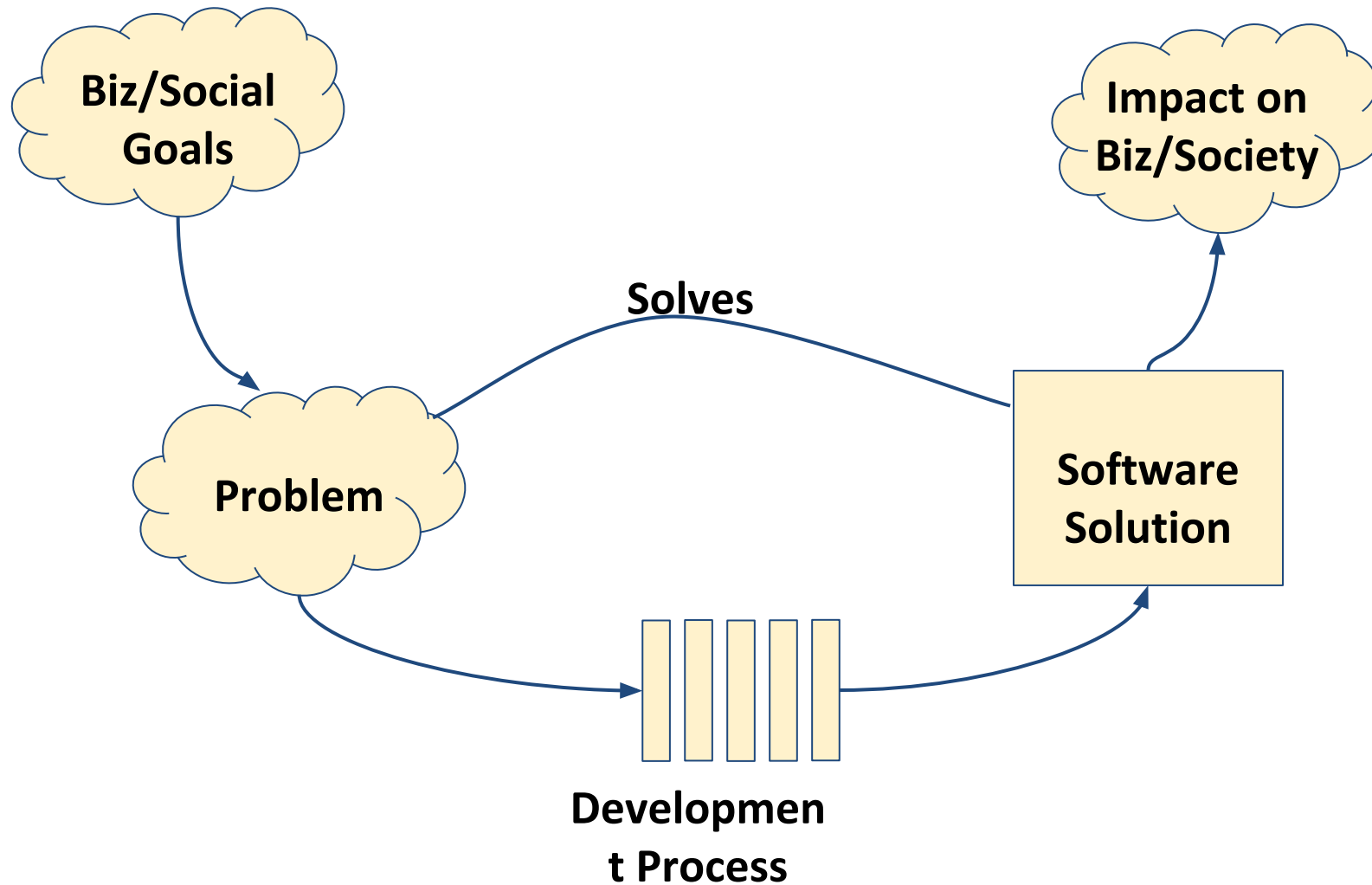
Background

- All systems, including software products/apps are built for some purpose, i.e. to achieve some goals
- Goals are higher level business/societal goals – improve efficiency, satisfaction, reduce cost, ...
- From these business/overall goals, specific requirements for the software are created
- Requirements state what the software should do (in order to help achieve business goals)
- Goals and requirements are different – goals are at a business level – the end impact of a software application, while requirements are for software

Software Solution Development



Context for Solution Development



IEEE Definition of Requirement

- All software projects start with some form of requirements specification
 - Every process model has some requirements analysis
 - The **rigor of initial requirement analysis differs**
- Identifying and specifying requirement **necessarily involves people interaction**; cannot be automated
- Requirement (**IEEE**)= A **condition or capability that must be possessed by a system**
- Req. phase ends with a software requirements specification (SRS) document
- SRS **specifies what the proposed system should do**

Problem Scale is an issue

- For small scale, understand and specifying requirements is easy
- For large problem - **very hard; probably the hardest, most problematic and error prone**
 - Input : user **needs in minds of people**
 - Output : **precise statement of what the future system will do**
- Requirements understanding is hard
 - **Visualizing a future system is difficult**
 - Capability of the future system not clear, hence needs not clear
 - **Requirements change with time**
 - ...

Essential to do a proper analysis and specification of requirements

Need for SRS

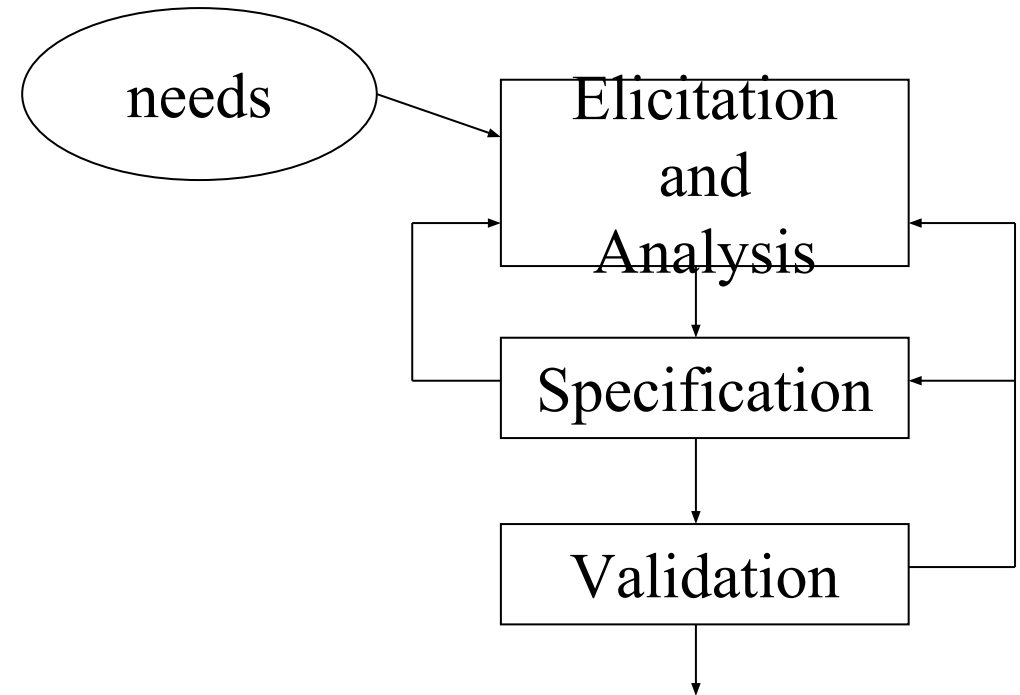
- Helps user understand users' needs
 - users do not always know their needs
 - must analyze and understand the potential
 - The requirement process helps clarify needs
- High quality SRS essential for high Quality SW
- SRS establishes basis of agreement between the user and the supplier
- SRS provides a reference for validation of the final product
 - Clear understanding about what is expected.
 - Validation - “ SW satisfies the SRS “
- Good SRS reduces the development cost
 - SRS errors are expensive to fix later
 - Good SRS can minimize changes and errors
 - Substantial savings; extra effort spent during req. saves multiple times that effort

Requirements Process

- Sequence of steps that need to be performed to convert user needs into SRS
- Process has to elicit needs and requirements and clearly specifies it
- Basic activities
 - ❖ problem or requirement analysis
 - ❖ requirement specification
 - ❖ validation
- Analysis involves elicitation and is the hardest

Requirements Process

- Process is not linear, **it is iterative and parallel**
- Overlap between phases - **some parts may be analyzed and specified**; specification itself may help analysis
- Validation can **show gaps that can lead to further analysis and spec**
- Divide and conquer is a common strategy (in solving all large problems):
 - decompose into small parts, understand each part and relation between parts



Problem Analysis and Requirement Elicitation

- Aim: to gain an understanding of the needs, requirements, and constraints on the software
- Analysis involves
 - interviewing client and users
 - reading manuals
 - studying current systems
 - helping client/users understand new possibilities
- Like becoming a consultant to the user
- Must understand the working of the organization , client and users

Problem Analysis and Requirement Elicitation

- Some issues
 - Obtaining the necessary information
 - Brainstorming: interacting with clients to establish desired properties
 - Information organization, as large amount of info. gets collected
 - Ensuring completeness
 - Ensuring consistency
 - Avoiding getting into design / coding issues
- Interpersonal and communication skills are important

Structured Approaches for Analysis

- Data flow analysis
- E-R diagram
- Object oriented analysis (and design)
- Use cases
- User stories
- Combination
- Formal modeling and specification
- ...

Informal Approach to Analysis

- Info obtained through analysis, observation, interaction, discussions,...
- No formal model of the system built – diagrams used as needed for understanding
- Obtained info organized in the SRS; SRS reviewed with clients and refined
- Relies on analyst experience and feedback from clients in reviews
- Goal: Specify the requirements in the SRS
- Useful in many/most contexts: we will use this

Suggested Process for Projecting the SRS

1. **Meet sponsor to understand the project**
 - What problem it is addressing (motivation / **biz goal**), the **main user categories**, what users in different categories need to be able to do with the application
 - Write this as the **scope/high level summary of the project** – a few paras (this will become **the first section of the SRS**)
2. *Estimate effort/size for SRS: **fill the pre form***
3. **Develop an initial SRS from this**
 - Give scope to LLM
 - Refine LLM's output
4. **Iterate as many times as needed**
 - Review SRS with sponsor (and users if possible)
 - Identify
 - (1) which requirements needs to be corrected, and
 - (2) missing requirements
 - Update the SRS – use LLMs as you see fit
5. *Capture effort/size/..: **fill the post form***

Pre and Post Information

- Pre-SRS form
 - Estimate the **size of the SRS**: # pages, # of use cases total for all user types
 - Estimate how many **hours of meeting with sponsors** / users will be needed
 - How many **person-hours will be needed to write the SRS**, if no LLM was available for use
- Post-SRS form
 - **Actual size** of the SRS: # pages, # of use cases
 - How many **actual hours of meeting with sponsors**
 - How many **actual person-hours spent** writing the SRS
 - How many **person-hours (in your view) you saved** in writing the SRS by using LLM
 - Which LLM did you use for SRS
 - Did you try other LLMs before finalizing one – if so which ones?
 - Did you set any context for the LLM – if so, what was it?
 - **How did you formulate your prompts** (can we give some guidance for answer this?)
 - Attach the **full interaction transcript**

Requirements Specification

- Final output of requirements analysis task is the SRS
- Transition from analysis to specification is not straight forward:
Brainstorming with the team
- knowledge about the system acquired in analysis used in specification
- Goal is to produce the SRS with some desired characteristics

Characteristics of an SRS

- **Correctness**
 - Each requirement accurately represents some desired feature in the final system
- **Completeness**
 - All desired features/characteristics must be specified
 - Hardest to satisfy
 - Completeness and correctness strongly related
- **Unambiguous**
 - Each requirement has exactly one meaning
 - Without this errors will creep in
 - Important as natural languages often used
- **Verifiability**
 - There **must exist a cost effective way of checking if software satisfies requirements**
- **Consistent**
 - two requirements don't contradict each other
- **Traceable**
 - The origin of the requirement, and how the requirement relates to software elements can be determined
- **Ranked for importance/stability**
 - Needed for prioritizing in construction
 - To reduce risks due to changing requirements

Components of an SRS

- What should an SRS contain ?
 - Clarifying this will help ensure completeness
- An SRS must specify requirements on
 - Functionality
 - Performance
 - Design constraints
 - External interfaces

Functional Requirements

- Heart of the SRS document; this forms the **bulk of the specs**
- **Specifies all the functionality** that the system should support for all categories of users
- **Outputs for the given inputs** and the relationship between them
- All operations the system is to do
- Should **specify behavior for invalid inputs too** - will help build a robust application

Performance Requirements

- All the performance constraints on the software system
- Generally on **response time , throughput** etc => **dynamic**
- **Capacity requirements** => **static**
- Must be in measurable terms (verifiability)
 - Eg resp time should be 90% of the time

Design Constraints

- Factors that restrict the choices – often from user / sponsor environment
- Some such restrictions
 - Choice of technology for implementation
 - Standard compliance and compatibility with other systems
 - Hardware Limitations
 - Reliability, fault tolerance, backup req.
 - Security and privacy

External Interface

- All interactions of the software with people, hardware, and software
- User interface most important
- These should also be verifiable, i.e. it should be possible to check if the software meets it

Specification Language

- Language should support desired char of the SRS
- Formal languages are **precise and unambiguous but hard**
- Natural languages mostly used, **with some structure for the document**
- Formal languages **used for special features or in highly critical systems**

For your project, think of a requirement and state it – Few functional and non-functional

Structure of SRS (for course project)

Will keep it short and precise – a few pages

- **Title** – the title of the project
- **Sponsor** - organization, contact name(s)
- **Introduction** (a few paras)
 - Purpose , the basic objective of the system; types of users of the sw
 - What functionality it provides to different user types
 - And important constraints (e.g. tech stack)
- **Functional requirements overview**
 - For each user category
 - Use cases for each user category – a table containing the use case name (explains the purpose), a short description, priority (high, medium, low)
- **Non-Functional requirements** (+ any other requirements)
 - User interface – key/common requirements
 - Performance requirements
 - Design constraints – tech stack to be used, security, ...
- **Further Details**
 - Details of each use case, of anything else, ...

Specifying Functional Requirements

- Functional requirements specify the intended behavior of the system, or what the system should do for the users
- Can be specified as services provided, tasks it can do, functions it supports, ...
- Some common methods of specifying them
 - List of software features / commands – directly specifies what functions it can perform
 - Use cases – specifies functionality from user perspective
 - User stories – similar to use cases
 - Others....

Use Cases

- Describe the **interaction of system with actors**
- **Actor**: External entity that interacts with system (could be a person, a system) – each user category can be an actor
- A use case should **specify a single function or single interaction of actor with system**
- A use case consist of:
 - A **name** that uniquely identifies it (goal)
 - The **actor** for whom this is a use case
 - Brief **description** (purpose/goal of the use case)
 - **Entry condition** for the use case – when performed
 - Main sequence of events to be performed (**interaction**)
 - **Exceptional sequence** of events

Use Cases

- A use case captures:
 - (i) who (actor),
 - (ii) does what (interaction, sequence) with the system, and
 - (iii) for what purpose (goal)
 - Also includes alternative sequences to satisfy the goal, or for exceptional / error situations
- It **does not say anything about system internals; the system is treated as a black box**; all interactions and responses are from outside the system perspective
- A **complete set of use cases specifies all the ways to use the system**, i.e. defines behaviours required of the system

Use Case Example

Start a quiz on the system

Actor: Instructor

This use case is for the instructor to start a quiz in the application for a course

Entry Condition: Instructor is logged in and has opened the course's page/screen

Event flow

1. Instructor chooses the type of quiz, and for multi-choice, selects the correct answer
2. Selects the duration of the quiz
3. Starts it
4. For math/text quizzes, species the answer
5. On expiry of the time, gets a summary of responses

Special/Exceptional Requirements

- Stop/cancel the quiz after starting

Linking Use Cases

- A use case **can use other use cases**, i.e. an use case may use others to achieve its goals
- Including a use case – use another use case
 - A step can use another use case – it should be specified by using **INCLUDE <use case no> in the step**
- Extend a use case - like in OO
 - Use case is defined as **EXTEND** some existing use case

Use Case tabular summary / overview

- Actor: Instructor

UC Name (Goal)	Brief Descr	Entry Condition	Exceptions (Names)
Start a quiz	Instructor starts a timed quiz	Instructor logged in	Cancel the quiz
Start a feedback	Instructor starts a timed feedback	same	Cancel
...			

- Actor: Student

UC Name (Goal)	Brief Descr	Entry Condition	Exceptions (Names)
Give response to the quiz	Enters his/her answer to the quiz	Quiz started	
...			

User Stories

- Another way to specify requirements
- Used in agile methods for software development
- Lightweight method of capturing who, what, why of an application requirement/feature
- A user story is very brief – a sentence or two.
- Each user story represents a requirement/feature
- User stories become assignable work items for software development

User Stories

- User story is generally of the form: "As a [type of user], I want [some action] so that [some reason / benefit]."
- Three parts to a user story:
 1. **Type of user**: Identify the specific type of user or stakeholder of the system. (Actor in UC)
 2. **Action**: Describe the desired action or functionality (to express actions in UC)
 3. **Benefit**: Explain the reason or benefit behind the user's goal. (Goal in UC)
- Collection of all user stories for an application can be the functional requirements of the appl.

User Stories

- As a customer, I want a shopping cart, so I can combine all my proposed purchases
- ALT: As an instructor, I want to start a quiz of different types, so I can give the exercise to students
- Enveave: As a project coordinator, I want to float a new project, so I can showcase what I am doing
- Enveave: As a project coordinator, I want to provide project updates, so as to share it with donors and others

Assuming SRS is written well and captures all the desired requirements, what could be a challenge with this SRS for developing the application?

Some Challenges with Requirements

- **Requirements bloating** (silver coating, ...): People ask for features that may be needed, but really don't get used
- **Requirements creep**. They keep increasing as the project progresses, new features added..
- For both, in SRS, **must rank each requirement** for importance (i.e. priority)
 - Essential,
 - desirable,
 - deferrable

Use Cases – Adding importance

UC Name (Goal)	Brief Descr	Entry Condition	Exceptions (Names)
Start a quiz (essential)	Instructor starts a timed quiz	Instructor logged in	Cancel the quiz
Start a feedback (essential)	Instructor starts a timed feedback	same	Cancel
Specify TA for getting quiz resp (desirable)	Instructor specifies a TA who will get the csv responses of the quiz		
Instructor can post a message (deferable)	Instructor posts a message on the application for all students		

Minimum Viable Product (MVP)

- For products / startups, where a product is conceived, the founders decide the requirements – what you think the users want
- Here **requirements bloating happens by envisioning more and more fancy features.**
- But the new product is inherently risky, user/ market understanding is limited.
- **Better to add features based on feedback from actual users;** but for feedback you need a working application/product
- Minimum viable product (MVP) is one of the most **important concepts for a startup venture**

MVP

- MVP concept became popular with lean startups - where the idea is to get the startup going with minimum resources in minimum time
- As entrepreneurs don't have a true idea of user needs, there is a tendency to pack everything - requirements bloat
- Idea is to build a working product quickly and then expand it based on user feedback
- This concept also applies to sponsored projects where sponsors/users are giving requirements, and followed by agile process

MVP

- An MVP is a product with just sufficient features to solve the main problem of users, i.e. they can use it for their goals
 - It is minimum and it is viable
- Purpose is to attract early-adopter customers and through their usage validate the product idea, its features, and what should be added to the product
- Goal: validate the product early by actual users
- MVP is a development technique for products and applications; used agile development

MVP

- New product is introduced in market with basic features – sufficient to get attention of users and for users to get value so they adopt it
 - This may be a beta release
- Full release may be done after getting feedback from users and refining the product/features
- Feedback of users through surveys, measurement through the sites of clicks, time spent, etc. used to refine or reposition the product

MVP

- Minimum features to provide value, and demonstrate future potential, to users so they adopt it
- But no one likes to adopt a product which is hard to use, unreliable, etc
- So a viable product should have
 - Sufficient features to provide value
 - Should be reliable
 - Should have good interface / usable
- MVP should be architected so features can be added, i.e. the design should be extensible

MVP and SRS

- In SRS, **all requirements should be prioritized**: Essential/Must have, Desirable, Optional
- Using this, **the collection of must-haves can be the starting point of an MVP**
- Examine these requirements
- **Remove those that can be deferred without making the product “unviable”**
- **Add those that must be needed**

Benefits of MVP approach

- **Fast release in market** of product; attract early adopters
 - MVP can be released in a couple of months
- **Validates product idea**, assumptions about users, with real-life data.
- **New feature releases based on real data**
- **Gets data on user behaviour to shape future product** initiatives and marketing strategy
- **Reduces waste** — save money and time that would otherwise be spent on fruitless ideas
- **Helps in fail-fast also** — if the product is not viable, better to fail early and fast and move on...

MVP use Example - Airbnb

- Airbnb. Wanted to create a market place offering personal housing for rent on a peer-to-peer basis
- Assumptions:
 - People will be willing to stay in stranger's homes and pay for bed, breakfast, and wifi
 - People will be willing to rent their private properties for short durations to strangers
- They used their own apartment to validate the idea
- Built a minimal website and published photos and other info about their apartment
 - Perhaps kept the rent low in the start to attract
- Found several paying guests soon – and through their usage / comments etc learned what users need, and through their experience learned what those posting properties need

MVP Examples

- Has been used widely. E.g.
 - Amazon
 - Groupon
 - Foursquare
 - Dropbox
 - Uber
 - Facebook
 - Instagram
- Standard practice now in lean startup
- Case studies available on internet

Requirements Validation

- Lot of room for misunderstanding
- Errors possible
- Expensive to fix requirement defects later
- Must try to remove most errors in SRS
- Most common errors
 - Omission - 30%
 - Inconsistency - 10-30%
 - Incorrect fact - 10-30%
 - Ambiguity - 5 -20%

Requirements Review

- SRS reviewed by a group of people
- Group: author, client, user, dev team rep.
- Must include client and a user
- Process – standard inspection process (read up on it – go over the document line by line and have each person OK it or give views)
- Effectiveness - can catch 40-80% of requirement errors

Summary

- Having a good quality SRS is important
- Requirements task **involve analysis, specification, and validation**
 - Analysis for problem modeling and understanding
 - Specification in SRS; natural language commonly used
 - Validation: **through reviews**
- Key properties of an SRS: **correctness, completeness, consistency, traceability, unambiguousness**
- Key components of an SRS: **Functionality, performance, interfaces, design constraints**